

Finding what nobody thought of: the four-oracle gap discovery method

A reusable method for discovering gaps a test plan cannot find, because the plan was written from the thing it is testing.

Run once, on the visual workflow builder, in July 2026. It produced 154 raw findings across four independent passes, merged to 103, of which 24 were approved for fixing and shipped. This document describes the method, what it cost, what it caught, and — the part worth reading — what it missed and why.

Provenance. The design is [docs-md/workflows/SPEC_COMPLETION_DESIGN.md](#). The pass outputs, merged register and decision sheets are in [feature-docs/20260724-workflow-builder-spec-completion/](#). This document is the method extracted from that specific run so it can be used again.

1. The problem it solves

We had a 90 KB manual test plan, 30 design documents, 108 unit specs and 25 Playwright specs. Bugs kept surfacing anyway — and not obscure ones. No undo. No copy/paste. No way to find a node in a large graph.

None of those appeared in the test plan. Not as gaps, not as deferred items, not as known discrepancies.

They couldn't. **The test plan was written from what got built.** It is a mirror, not an oracle: it can confirm the implementation matches itself, and it is structurally incapable of failing on something nobody thought of. Add a feature and you add its tests; forget a feature and you forget its tests too, and nothing anywhere registers the absence.

This is the defining property of any specification derived from an implementation, and no amount of diligence inside that specification fixes it. You need a source of truth that was not derived from the code.

1.1 The rejected alternative

The obvious proposal was: take every bug we've fixed, generalise it into an invariant, audit every surface against it.

Rejected as the primary strategy. Invariants derived from fixed bugs are drawn from a biased sample — the bugs that happened to be hit on the paths that happened to be walked. They find *siblings of known bugs* efficiently, and they are structurally incapable of surfacing a capability nobody has considered. "There is no undo" is not the sibling of any bug.

Invariants kept a role, demoted: as a **regression floor** after the fact, one cross-surface check per gap class closed. That is what they are good at.

2. The core idea

An oracle is a source of expected behaviour that is independent of the implementation. The method runs four of them, in parallel, each along a different axis, none able to see the others' output.

Two structural choices make it work.

Discover first, specify second. The conventional order — write the full spec, diff it against the implementation — costs weeks re-describing behaviour that existing documents already describe, and a spec written from the same head that built the system reproduces the same blind spots. Inverting it means the passes tell you *where* specification is missing, and you write spec only for that territory. It is the difference between a two-week effort and a two-month one.

Independence is the whole asset. The four passes must not share findings, vocabulary drift aside. When two passes that could not see each other land on the same gap, that agreement is the strongest priority signal available — stronger than any severity rating a single author assigns.

3. The four oracles

Each ran as one subagent against a shared inventory, with a fixed output schema. **Discovery only — no code changes, no fixes.**

Pass B — Editor-environment obligations

Axis: what environments of this class are expected to do.

The builder is a programming environment. Programming environments carry a known duty roster, and you can audit any surface against it without knowing anything about how this one was built:

CRUD · undo/redo · duplicate · copy/paste · multi-select · find & navigate · refactor (rename, extract, inline) · inspect & debug · error recovery · diff & compare · concurrent editing · keyboard access

For each obligation on each surface: **present / partial / absent**, with evidence. Absent-and-reasonably-expected is a finding.

Cheapest oracle, highest yield per hour. A 90-second probe confirmed three hits before the pass formally started. It is the only one of the four that reaches *capability-level* absences — the things missing so completely that no bug can point at them.

Pass D — Mutation and cascade

Axis: what happens when something upstream changes or disappears.

Owens the change/delete axis outright. For every dependency edge in the domain model — node→binding, binding→variable, group→member, catalog→port kind, version→reference, run→cache — specify what happens when the upstream mutates or vanishes.

A finding is any edge where behaviour is undefined, silently destructive, or leaves the artifact invalid with no recovery path.

This is where "stumble" bugs live. Before the method ran, cascade bugs were being discovered one at a time, each as a surprise, because no table of dependency edges existed to check systematically.

Pass A — Author journeys

Axis: can a competent person actually get their job done.

Six to eight goal-first journeys drafted from **real workloads**, written *before* consulting the UI, then walked against it. A finding is any wall, and any step a competent author would not guess.

This pass requires human input and fails without it. Journeys invented by the same person who will walk them are plausible fiction, and a walk against fiction is worthless. They must be red-penned by someone who knows the real work before the walk begins.

Highest blocker density of the four: 11 blockers from 38 findings.

Pass C — Domain cross-product

Axis: every combination of states, enumerated.

Enumerate and mark every cell **specified / unspecified / won't-support** across the product's state dimensions — control-flow nesting combinations, port-kind × binding-state, run-status × every surface that renders it, validation-severity × anchor target.

Scoped explicitly **off** the mutation axis, which D owns. Without that exclusion the two passes return the same findings in different words and the independence is fake.

Highest volume, lowest density: 52 findings, one blocker. Its distinctive output is not the bugs — it's the **won't-support** cells, which become a written non-goals register. That is worth as much as a fix, because it stops the same non-decision being rediscovered by a different person six months later.

3.1 Choosing axes for a different system

The four above are specific to an interactive authoring tool. The property that matters is that **each axis reaches findings the others structurally cannot**. For a different system, pick axes with that same disjointness:

Axis	Question it asks
Class obligations	What does anything of this kind owe its user?
Mutation and cascade	What breaks when an upstream thing changes or dies?
Goal journeys	Can a real person finish a real task?
State cross-product	Which combinations has nobody decided about?

Two axes that overlap are one axis and a waste of a pass.

4. The machinery that makes it merge

Four passes producing four essays is not a result. Three pieces of scaffolding turn them into one ranked register.

4.1 A shared inventory, built first, in one head

Artifacts, surfaces, dependency edges, state sources — named once, before any pass runs. Naming is authoritative and every pass cites it.

Four passes inventing their own vocabulary produce findings that cannot be merged, and you discover that only after paying for all four.

4.2 A fixed finding schema

```
id          stable identifier, prefixed by pass
pass        A | B | C | D
title       one line
severity    blocker | major | minor
type        design-gap | impl-gap | non-goal
evidence    file:line, or reproduction steps
surfaces    affected surfaces, in inventory vocabulary
disposition proposed: fix | defer | won't-support
rationale   why that disposition
```

Emitted as JSON. The merge is then mechanical rather than editorial.

4.3 Merge that preserves disagreement

Every source finding lands in exactly one merged entry; nothing is dropped and nothing is silently re-adjudicated. Where merged sources proposed different dispositions, **the disagreement is stated rather than resolved** — resolving it is the gate's job, not the merger's.

Merged severity is the *maximum* across the cluster.

5. The disposition gate

The passes will find more than is worth building. Without a gate, the effort has no natural end.

Every finding is dispositioned **fix / defer / won't-support** — recommended by whoever implements, **approved by a human who owns the scope**. `defer` and `won't-support` are first-class outcomes, not failures.

In practice the gate ran over the 27 entries that were either a blocker or corroborated by two or more passes. Three clusters carried a genuine scope judgement and were ruled on individually; everything else kept the disposition its pass proposed. Net: **24 fix, 3 defer**.

That ratio is the point. Discovery is cheap and generous; the gate is where the size of everything downstream is decided.

6. What it actually produced

Pass	Findings	Blocker	Major	Minor
A — journeys	38	11	18	9
B — obligations	27	3	15	9
C — cross-product	52	1	25	26
D — cascade	37	5	26	6
Source total	154	20	84	50
Merged	103	16	56	31

154 source findings merged to 103 entries: 86 single-pass, **17 corroborated by two or more passes**, one by three. **None by all four**.

That last number is the design working. The axes were disjoint enough that a 4-way hit was never likely — which is exactly what you want, and also means corroboration is a *sparse* signal. Seventeen out of 103 is enough to rank the top of the register and no more. Don't plan on it doing more than that.

Among the corroborated blockers: `errorPolicy` had no authoring surface at all — one of three modelled edge flavours was unreachable from the product, and a config that arrived with it set could never be saved again. Three passes found it independently, by three different routes.

7. What it missed, and why that matters more

The stated acceptance criterion was: *a manual walkthrough afterwards produces zero functional findings, only subjective polish.*

It did not hold. A walkthrough three weeks later produced nine further defects, several inside the scope the four passes had covered.

The most instructive is the pair **G-021 / D-17**.

Pass discovery found G-021: the cancel-on-new-Try predicate needed a discriminator so that starting a new preview wouldn't cancel unrelated production runs. Correct finding. It was fixed: the backend gained a `RunTrigger` search attribute, the cancel predicate narrowed to `RunTrigger = "try"`, and unit tests covered both sides.

Three weeks later a live walkthrough started two previews on a slow graph and watched both run to completion. **Nothing in the product ever produced a run stamped "try"**. The editor posted to the general run endpoint, which hard-codes `"api"`. The cancel ran on every start and always swept an empty set. G-021's careful narrowing was protecting a category that never got created.

Every static oracle looked at that code and saw it was correct — because each half *was* correct. What none of them could see is that the two halves never met.

7.1 The limit this reveals

The four oracles all read the system as written. That reaches:

- absences (B),
- undefined transitions (D),
- unwalkable paths (A),
- undecided combinations (C).

It does not reach **seams**: two components each correct in isolation, disagreeing about the contract between them. A seam has no missing code to find, no undefined state to enumerate, no absent capability to name. It is only observable when both sides run at once and the result is wrong.

Three of the nine walkthrough defects were seam defects, all in the same family: a frontend and a backend each self-consistent, each unit-tested, each wrong about the other.

The remedy is not a fifth static pass. It is to treat *running the thing and watching it* as an instrument of equal standing to the four oracles, budgeted and scheduled rather than done at the end if time permits. The two instruments find different classes of defect and neither substitutes for the other.

7.2 Two smaller lessons, both cheap to avoid

A gap register decays, fast. By the time the second gate ran, 261 commits had landed since the register was written and only 30 of them named a gap id. Sampling four entries found two already fixed by work that never mentioned them. **Verify before ruling** — asking someone to decide about a gap that no longer exists wastes the scarcest resource in the process. Verification and ruling are two jobs; conflating them is a false economy.

A pass can produce a confident false finding. One finding — "the active-edge animation is dead code" — was withdrawn during re-verification. It rested on a `grep` that returned empty because the tool classified the file as binary (it contained two NUL bytes) and silently declined to search it. A silent-empty search is indistinguishable from a real absence, and "absence" is precisely what these passes are hunting. Use `grep -a`, and treat any finding whose whole evidence is *"I searched and found nothing"* as unconfirmed until something positive corroborates it.

8. Using it again

The sequence, with the effort concentrated where it belongs:

1. **Build the shared inventory** — one head, one page. Artifacts, surfaces, dependency edges, state sources.
2. **Draft the journey scripts** and get them red-penned by someone who knows the real work. This is the long pole; start it first.
3. **Fan out the passes that don't need the red-pen** (obligations, cross-product, cascade) in parallel against the inventory.
4. **Run the journey pass** once the scripts are corrected.
5. **Merge mechanically** on the fixed schema. Rank by corroboration, then severity, then breadth. Preserve disagreements.
6. **Verify, then gate.** Two separate jobs, in that order. `defer` and `won't-support` are wins.
7. **Fix, with a regression floor** — one cross-surface check per gap *class*, so the class cannot silently reopen.
8. **Walk it live.** Budget this as an instrument, not a formality. Expect it to find things all four passes could not, and treat each such find as information about which axis was missing.

When it's worth it. When bugs keep arriving that the test suite had no way to catch, and the suite's own coverage numbers look fine. That combination is the signature of a specification derived from its implementation, and it is the only condition under which this method pays for itself.

When it isn't. On a system small enough to hold in one head, or young enough that the specification is still ahead of the code. The method's cost is mostly in the inventory and the merge, and both scale with surface count.